60-DAY JOB READY PLAN

Md Zuber · Senior Java Backend Engineer

Core Java · Collections (80 Q&A) · Spring Boot · Microservices · Kafka · AWS · SQL · System Design

Your Starting Strengths

Based on your resume, you already have solid production experience at Deloitte (JP Morgan project) and Infosys (VISA RTP system). This plan is designed to fill gaps, deepen interview-level knowledge, and give you 80 curated Q&A for collections alone.

4+ Yrs Experience | **Spring Boot ✓** | **AWS Production ✓** | **Kafka Experience ✓**

60-Day Master Schedule

Phase	Days	Focus Area
Phase 1	Days 1–10	Core Java Deep Dive + Java 8/17 Features
Phase 2	Days 11–18	Collections Framework + 80 Interview Q&A
Phase 3	Days 19–26	Spring Boot Advanced + Testing
Phase 4	Days 27–34	Microservices Architecture + Patterns
Phase 5	Days 35–42	Apache Kafka Deep Dive
Phase 6	Days 43–50	AWS Deep Dive (S3, EC2, CloudWatch, Terraform)
Phase 7	Days 51–56	SQL Mastery + Design Patterns
Phase 8	Days 57–60	System Design + Mock Interviews + Resume Polish

PHASE 1 · DAYS 1–10

Core Java Deep Dive

Days 1–2: OOP Principles & Memory Model

Topics to Master

- SOLID Principles with real code examples (you'll be asked to explain these)
- JVM Architecture: Stack vs Heap, Method Area, GC Roots
- String Pool, String interning, == vs .equals()
- final, finally, finalize — differences and use cases
- static keyword: static blocks, static initializers, static imports
- Wrapper classes, auto-boxing, unboxing pitfalls (Integer cache -128 to 127)

Top Interview Questions — OOP & Memory

Q: What is the difference between == and .equals() for String?

== compares references (memory address). .equals() compares character content. New String("abc") == new String("abc") is false, but .equals() is true. String literals use the pool so "abc" == "abc" can be true. Always use .equals() for content comparison.

Q: Explain method overloading vs overriding. What are the rules?

Overloading: same name, different parameters, resolved at compile time (static polymorphism). Overriding: same signature in subclass, resolved at runtime (dynamic polymorphism). Rules for overriding: cannot reduce visibility, cannot throw broader checked exceptions, return type must be same or covariant, cannot override static/final/private methods.

Q: What is the purpose of the 'transient' keyword?

Marks a field to be excluded from Java serialization. When an object is serialized, transient fields are not written to the stream. On deserialization, they get their default values (null for objects, 0 for numbers, false for boolean). Used for sensitive data (passwords) or non-serializable fields.

Days 3–4: Java 8 Features (Critical for Interviews)

Must-Know Java 8 Topics

- Lambda expressions: syntax, functional interfaces, method references
- Stream API: map, filter, reduce, collect, flatMap, distinct, sorted, limit
- Optional<T>: orElse, orElseGet, ifPresent, map, filter — avoid NullPointerException
- Default and static methods in interfaces
- Predicate<T>, Function<T,R>, Consumer<T>, Supplier<T> — built-in functional interfaces
- Collectors: toList, groupingBy, partitioningBy, joining, counting
- Date/Time API: LocalDate, LocalDateTime, ZonedDateTime, Duration, Period

Java 8 Code Patterns You Must Know

Stream Chaining — filter + map + collect

```
List<String> names = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(Employee::getName)
    .sorted()
    .collect(Collectors.toList());
```

GroupingBy — very common in interviews

```
Map<String, List<Employee>> byDept =
    employees.stream().collect(Collectors.groupingBy(Employee::getDept));
```

Optional — avoid NPE

```
Optional.ofNullable(user).map(User::getEmail).orElse("N/A");
```

Days 5–6: Java 11–17 Features

Java 11–17 Key Features

- Java 11: var in lambda, new String methods (isBlank, strip, lines, repeat), Files.readString/writeString
- Java 14: Records (preview) — immutable data carriers
- Java 15: Text Blocks — multi-line strings with """
- Java 16: Records (final), Pattern Matching for instanceof
- Java 17 (LTS): Sealed classes, Pattern Matching switch (preview), enhanced switch expressions
- Switch expressions: return a value, use yield keyword
- NullPointerException improvements — helpful NPE messages

Q: What are Java Records and when do you use them?

Records (Java 16+) are immutable data classes. The compiler auto-generates constructor, getters, equals, hashCode, toString. Use for DTOs, value objects, or any class that simply holds data. Example: record Point(int x, int y) {} — you get Point.x(), Point.y(), equals(), hashCode() and toString() for free. They cannot extend other classes (implicitly extend Record) but can implement interfaces.

Days 7–8: Multithreading & Concurrency

Concurrency Topics (Frequently Asked)

- Thread lifecycle: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED
- Runnable vs Callable vs Thread — differences and use cases

- synchronized keyword: method-level vs block-level
- volatile keyword: visibility guarantee, happens-before relationship
- wait(), notify(), notifyAll() — object-level inter-thread communication
- ExecutorService, ThreadPoolExecutor, FixedThreadPool, CachedThreadPool
- Future<T> and CompletableFuture — async programming
- ReentrantLock, ReadWriteLock, StampedLock
- Atomic classes: AtomicInteger, AtomicReference, AtomicLong
- CountDownLatch, CyclicBarrier, Semaphore, Phaser
- Deadlock: causes, detection, prevention strategies
- ThreadLocal<T> — per-thread data without synchronization

Q: What is the difference between synchronized and ReentrantLock?

synchronized is simpler but limited: JVM manages lock, no timeout, no condition variables, not interruptible. ReentrantLock is explicit: tryLock(timeout), lockInterruptibly(), multiple Condition objects (await/signal), fairness policy option. Use ReentrantLock when you need timed locks, interruptible locking, or multiple wait sets. synchronized is preferred for simple cases (less error-prone, JVM can optimize).

Q: Explain CompletableFuture and its advantages over Future.

Future.get() blocks the calling thread. CompletableFuture supports non-blocking: thenApply (transform result), thenAccept (consume result), thenCombine (combine two futures), exceptionally (handle errors), allOf/anyOf (coordinate multiple futures). It enables functional-style async pipelines and can be completed manually. Much more powerful for reactive-style code without blocking.

Days 9–10: Exception Handling + Design Principles

Exception Handling Best Practices

- Checked vs Unchecked exceptions — when to use each
- Custom exceptions: extending RuntimeException vs Exception
- try-with-resources — AutoCloseable interface
- Multi-catch syntax: catch (IOException | SQLException e)
- Exception chaining: new RuntimeException("msg", cause)
- Don't swallow exceptions — always log or rethrow
- Global exception handlers with @ControllerAdvice in Spring

PHASE 2 · DAYS 11–18

Collections Framework — 80 Interview Q&A

Collections Overview

Collection Hierarchy Quick Reference

List	Set	Queue/Deque	Map
ArrayList LinkedList Vector Stack CopyOnWriteArrayList	HashSet LinkedHashSet TreeSet EnumSet CopyOnWriteArraySet	PriorityQueue LinkedList ArrayDeque ConcurrentLinkedQueue	HashMap LinkedHashMap TreeMap ConcurrentHashMap EnumMap

Section 1: ArrayList & LinkedList (Q1–Q12)

Q1: What is the internal data structure of ArrayList?

ArrayList uses a dynamic resizable array (Object[]). Default capacity is 10. When full, it creates a new array of 1.5x the size and copies all elements. Random access is O(1). Insert/delete in middle is O(n) due to shifting.

Q2: When does ArrayList resize and what is the growth factor?

When size == capacity and you add a new element. Growth formula: newCapacity = oldCapacity + (oldCapacity >> 1), effectively 1.5x. Use ensureCapacity(n) or new ArrayList<>(initialCapacity) to avoid frequent resizing for known large datasets.

Q3: ArrayList vs LinkedList — when to use which?

ArrayList: faster random access O(1), better cache locality, less memory overhead. Use for frequent reads. LinkedList: faster add/remove at head/tail O(1), implements Deque so usable as stack/queue. Use when frequent insertions/deletions at both ends. LinkedList uses more memory (node pointers).

Q4: Is ArrayList thread-safe? How to make it thread-safe?

Not thread-safe. Options: 1) Collections.synchronizedList(new ArrayList<>()) — wraps each method in synchronized block. 2) CopyOnWriteArrayList — creates a fresh copy on every write, reads never block; ideal for read-heavy scenarios. 3) Use Vector (legacy, not recommended).

Q5: What happens if you modify a List while iterating with a for-each loop?

ConcurrentModificationException is thrown. The iterator checks a modCount field. Use Iterator.remove() for safe removal during iteration, or use removeIf() lambda, or iterate a copy, or use CopyOnWriteArrayList.

Q6: How does contains() work in ArrayList vs LinkedList?

Both use O(n) linear scan calling equals(). ArrayList has better cache performance. Use a HashSet for O(1) contains checks when that's the primary operation.

Q7: What is the difference between remove(int index) and remove(Object o) in ArrayList?

remove(int index) removes element at that position, O(n) due to shifting. remove(Object o) removes first occurrence by equals(), also O(n). Be careful with Integer: list.remove(1) calls remove(int), list.remove(Integer.valueOf(1)) calls remove(Object).

Q8: How to sort an ArrayList?

Collections.sort(list) — uses merge sort (TimSort), O(n log n), stable. list.sort(Comparator) — Java 8+. For custom objects: list.sort(Comparator.comparing(Employee::getSalary).thenComparing(Employee::getName)).

Q9: What is the time complexity of get(), add(), remove() in LinkedList?

get(index) is O(n) — traverses from head or tail (whichever is closer). add()/addLast() O(1). addFirst() O(1). remove() from middle O(n) to find, O(1) to unlink. This is why LinkedList is rarely the best choice for indexed access.

Q10: How does subList() work in ArrayList?

subList(from, to) returns a view (not a copy) of the original list. Changes to the subList reflect in the original and vice versa. If the original list is structurally modified after subList(), a ConcurrentModificationException will be thrown on subList access. To get an independent copy: new ArrayList<>(list.subList(1, 4)).

Q11: What is Collections.unmodifiableList() and when to use it?

Returns a view that throws UnsupportedOperationException on mutation operations. Note: the underlying list can still be modified by the original reference — it only prevents modification via the unmodifiable view. For truly immutable: List.of() (Java 9+) or List.copyOf() — null-free, truly immutable.

Q12: Explain the difference between Arrays.asList() and List.of().

Arrays.asList(): fixed-size but mutable (set allowed, add/remove not); backed by array; allows nulls; returns java.util.Arrays$ArrayList. List.of() (Java 9+): completely immutable (no set/add/remove); does NOT allow nulls; not backed by an array. For most modern code, prefer List.of() for constants.

Section 2: HashMap & Hashing (Q13–Q30) — Most Asked Topic

Q13: How does HashMap work internally in Java 8+?

Uses array of Node<K,V> buckets. Index = hash(key) & (capacity-1). Each bucket holds a linked list; when bucket size > 8 and table size >= 64, it converts to a Red-Black Tree (treeify) for O(log

n) worst case. Default capacity 16, load factor 0.75. When size > capacity * loadFactor, it doubles and rehashes.

Q14: What happens when two keys have the same hashCode?

Hash collision: both entries go to the same bucket and form a linked list (chaining). Retrieval: traverse the list calling equals() to find the right key. This is why overriding hashCode() alone is not enough — equals() must also be consistent.

Q15: Why must you override both hashCode() and equals() together?

Contract: if a.equals(b) is true, then a.hashCode() == b.hashCode() MUST be true. If you override only equals(), two equal objects can land in different buckets — HashMap will never find the value. If you override only hashCode(), objects in the same bucket won't match via equals() — duplicates can be inserted.

Q16: What is the load factor in HashMap and how does it affect performance?

Load factor = size/capacity threshold for rehashing. Default 0.75 balances memory vs time. Lower load factor (e.g., 0.5): fewer collisions, more memory, more frequent resizing. Higher (e.g., 0.9): more collisions, less memory. HashMap(initialCapacity, loadFactor) for tuning.

Q17: HashMap vs LinkedHashMap vs TreeMap — differences?

HashMap: O(1) average, no order guarantee. LinkedHashMap: O(1), maintains insertion order (or access order for LRU cache). TreeMap: O(log n), sorted by natural order or Comparator. Use TreeMap for sorted key access, LinkedHashMap for ordered iteration or LRU cache.

Q18: How to implement an LRU Cache using LinkedHashMap?

Extend LinkedHashMap with accessOrder=true in constructor, override removeEldestEntry() to return true when size > maxSize. Example: new LinkedHashMap<>(capacity, 0.75f, true) { protected boolean removeEldestEntry(Map.Entry<K,V> e) { return size() > maxSize; } }

Q19: What is ConcurrentHashMap and how is it thread-safe?

Uses segment-level (Java 7) or bucket-level CAS+synchronized (Java 8+) locking. In Java 8, only the affected bucket is locked during write, allowing higher concurrency. Read operations are non-blocking. Null keys and values are NOT allowed (unlike HashMap). computeIfAbsent(), merge() are atomic operations.

Q20: HashMap vs Hashtable vs ConcurrentHashMap?

Hashtable: legacy, synchronizes every method (entire map), slow, no nulls. HashMap: not thread-safe, allows one null key. ConcurrentHashMap: fine-grained locking, high concurrency, no null keys/values. Always prefer ConcurrentHashMap over Hashtable. Never use Hashtable in new code.

Q21: What is the worst case time complexity for HashMap?

O(n) in Java 7 (all keys in same bucket as linked list). O(log n) in Java 8+ when bucket is treeified. Caused by hash collision attacks or poor hashCode implementations. This is why Java 8 introduced treeification.

Q22: Can we use a mutable object as a HashMap key?

Technically yes, but strongly discouraged. If the key's fields change after insertion, hashCode() changes, the key falls in a different bucket, and you can never retrieve the value. Best practice: use immutable objects (String, Integer, custom immutable classes) as keys.

Q23: What is HashMap's behavior with null keys?

HashMap allows exactly one null key, stored at bucket index 0. Hashtable and ConcurrentHashMap do NOT allow null keys. TreeMap does not allow null keys (NullPointerException on comparison).

Q24: Explain Map.Entry and how to iterate a HashMap.

Map.Entry is an inner interface representing a key-value pair. Iteration options: 1) entrySet() for-each (most efficient). 2) keySet() then get(key) (two lookups, less efficient). 3) values() to iterate only values. 4) forEach((k,v) -> ...) with Java 8 lambda. Always prefer entrySet() for key+value iteration.

Q25: What is getOrDefault(), putIfAbsent(), computeIfAbsent()?

getOrDefault(key, default): returns value or default without inserting. putIfAbsent(key, val): inserts only if key absent, thread-safe in ConcurrentHashMap. computeIfAbsent(key, fn): inserts fn.apply(key) if absent, returns existing or new value. Great for building Map<K, List<V>>: map.computeIfAbsent(key, k -> new ArrayList<>()).add(value).

Q26: How does HashMap handle rehashing?

When size exceeds capacity * loadFactor, capacity doubles. All existing entries are rehashed (new index = hash & newCapacity-1). This is O(n) operation. In Java 8, optimization: entries either stay at same index or move to index + oldCapacity — no need to recompute hash for each entry during resize.

Q27: What is WeakHashMap and when to use it?

Uses WeakReferences for keys. When a key has no strong references outside the map, GC can collect it, and the entry is automatically removed. Use case: caches, metadata about objects where the map shouldn't prevent GC. ThreadLocal's internal implementation uses a WeakReference for thread-to-value mapping.

Q28: How to make a HashMap thread-safe without ConcurrentHashMap?

1) Collections.synchronizedMap(new HashMap<>()) — synchronized on the entire map, all operations locked. Must manually synchronize iteration: synchronized(map) { for (Map.Entry e : map.entrySet()) {...} }. 2) ConcurrentHashMap is almost always better — higher concurrency.

Q29: What is EnumMap and why is it faster than HashMap for enum keys?

EnumMap uses a plain array indexed by the enum's ordinal. O(1) access with no hashing needed, no collision handling, extremely memory efficient. Always use EnumMap when keys are an enum type. EnumSet similarly uses a bitmask for O(1) operations.

Q30: Explain TreeMap's internal data structure and ordering.

TreeMap uses a Red-Black Tree (self-balancing BST). All operations (get, put, remove) are O(log n). Keys must implement Comparable or you must provide a Comparator. headMap(), tailMap(), subMap(), floorKey(), ceilingKey() are powerful range operations unavailable in HashMap.

Section 3: HashSet & Other Sets (Q31–Q40)

Q31: How does HashSet work internally?

HashSet is backed by a HashMap. When you add(element), it calls map.put(element, PRESENT) where PRESENT is a static dummy Object. All HashMap rules apply: hashCode() + equals() determines uniqueness. Contains() is O(1) average.

Q32: HashSet vs LinkedHashSet vs TreeSet?

HashSet: O(1) add/remove/contains, no order. LinkedHashSet: O(1), maintains insertion order. TreeSet: O(log n), sorted natural order or Comparator, implements NavigableSet. Use TreeSet for sorted unique elements, or when you need floor/ceiling/headSet/tailSet operations.

Q33: How to remove duplicates from a List while preserving order?

new LinkedHashSet<>(list) preserves insertion order and removes duplicates. Then new ArrayList<>(linkedHashSet) to convert back. Java 8: list.stream().distinct().collect(Collectors.toList()) also works (uses equals/hashCode).

Q34: What are Set operations: union, intersection, difference?

Union: Set<T> union = new HashSet<>(setA); union.addAll(setB). Intersection: Set<T> inter = new HashSet<>(setA); inter.retainAll(setB). Difference: Set<T> diff = new HashSet<>(setA); diff.removeAll(setB). These mutate the copy, not the originals.

Q35: Can HashSet contain null?

HashSet can contain one null (since HashMap allows one null key). TreeSet cannot contain null (comparison would throw NPE). LinkedHashSet can contain one null. ConcurrentSkipListSet cannot contain null.

Q36: What is CopyOnWriteArraySet?

Backed by a CopyOnWriteArrayList. Thread-safe, all mutating operations create a new copy of the internal array. Reads are non-blocking. Suitable for small sets with infrequent writes and frequent reads/iterations. High memory overhead for write-heavy scenarios.

Q37: How does TreeSet handle custom sorting?

Pass a Comparator to the constructor: new TreeSet<>(Comparator.comparing(Employee::getSalary)). The comparator defines BOTH ordering and equality for TreeSet — if compareTo returns 0, elements are considered equal (duplicate) even if equals() says different. This can cause surprising deduplication!

Q38: What is NavigableSet and its key methods?

NavigableSet (TreeSet implements it): floor(e) — greatest element <= e; ceiling(e) — least element >= e; lower(e) — strictly less; higher(e) — strictly greater; pollFirst()/pollLast() — retrieve and remove. descendingSet() — reverse view. Very useful for range-based problems.

Q39: How does equals() and hashCode() affect Set behavior?

If two objects are equal (equals() returns true), they must have the same hashCode(). Adding both to a HashSet will result in only one entry. If hashCode() is inconsistent (returns different values for equal objects), both will be added — breaking the Set contract. Always generate both methods together (IDE or Lombok @EqualsAndHashCode).

Q40: What is a BitSet and when would you use it?

BitSet is a memory-efficient set of bits (booleans). Operations: set(index), get(index), and(), or(), xor(), cardinality() (count of set bits). Use for large boolean arrays, sieve of Eratosthenes, flags, bloom filters, and anywhere you need a compact representation of a set of integers.

Section 4: Queue, Deque & Stack (Q41–Q50)

Q41: What is the difference between Queue methods: add/offer, remove/poll, element/peek?

add() throws exception if queue full; offer() returns false. remove() throws NoSuchElementException if empty; poll() returns null. element() throws if empty; peek() returns null. Always prefer offer/poll/peek for safer code. These pairs exist for bounded vs unbounded queues.

Q42: When to use ArrayDeque vs LinkedList as a Stack/Queue?

ArrayDeque is faster and has better memory locality (no node overhead). Implements Deque, works as both stack (push/pop) and queue (offer/poll). LinkedList has null support and implements List too, but is slower. Java docs recommend ArrayDeque over Stack class. Stack class is legacy (extends Vector).

Q43: How does PriorityQueue work?

Min-heap by default (smallest element at head). Internally uses an array-based binary heap. offer()/add() and poll() are O(log n). peek() is O(1). For max-heap: new PriorityQueue<>(Collections.reverseOrder()). For custom: new PriorityQueue<>(Comparator.comparing(Task::getPriority)). Contains() is O(n).

Q44: What is BlockingQueue and its common implementations?

BlockingQueue blocks on take() if empty, and put() if full — used for producer-consumer patterns. ArrayBlockingQueue: bounded, backed by array. LinkedBlockingQueue: optionally bounded. PriorityBlockingQueue: unbounded, ordered. DelayQueue: elements available after delay. SynchronousQueue: no internal buffer, handoff between threads.

Q45: How to implement a producer-consumer using BlockingQueue?

Producer: queue.put(item) — blocks if full. Consumer: item = queue.take() — blocks if empty. No manual synchronization needed. Example: BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100). This is the standard pattern for thread pools and async task processing.

Q46: Difference between Deque and Queue?

Queue is single-ended (FIFO). Deque (double-ended queue) supports add/remove at both ends. As stack: push()/pop() at head. As queue: offer()/poll() at tail/head. ArrayDeque implements Deque and is the recommended implementation. LinkedList also implements Deque but slower.

Q47: What is ConcurrentLinkedQueue?

Non-blocking, thread-safe queue using CAS (Compare-And-Swap) operations. No locking — very high throughput. Does not support blocking (use LinkedBlockingQueue for blocking). size() is O(n) — not constant time. Best for high-throughput, non-blocking concurrent access.

Q48: How to find the kth largest element using a PriorityQueue?

Maintain a min-heap of size k. Iterate all elements: if current > heap.peek(), poll and offer current. After full iteration, heap.peek() is the kth largest. Time: O(n log k). More efficient than sorting: O(n log n). This pattern appears frequently in top-k problems.

Q49: What is a monotonic stack and when is it useful?

A stack that maintains elements in strictly increasing or decreasing order. As you push, pop elements that violate the monotonic property. Use for: next greater element, next smaller element, largest rectangle in histogram, stock span problems. Time complexity: O(n) because each element is pushed and popped at most once.

Q50: Explain TransferQueue and its use case.

TransferQueue (implemented by LinkedTransferQueue) extends BlockingQueue. Key method: transfer(e) — blocks until a consumer takes the element (unlike put() which just adds to the queue). tryTransfer(e) returns immediately. Use when the producer needs confirmation that a consumer has received the item.

Section 5: Iterator, Comparator & Utility Methods (Q51–Q60)

Q51: What is the difference between Comparable and Comparator?

Comparable: implemented by the class itself, defines natural ordering via compareTo(). Single ordering only. Comparator: external, defines custom ordering via compare(). Multiple orderings possible. Comparator.comparing() chains:

Comparator.comparing(Person::getAge).thenComparing(Person::getName). Use Comparable for one canonical order, Comparator for flexibility.

Q52: How does Collections.sort() decide the order for a custom class?

Either the class implements Comparable (natural order), or you pass a Comparator. If neither, ClassCastException at runtime. list.sort(null) uses Comparable. Internally uses TimSort — O(n log n) stable sort, efficient on partially sorted data.

Q53: What is fail-fast vs fail-safe iterator?

Fail-fast: throws ConcurrentModificationException if the collection is structurally modified during iteration (ArrayList, HashMap iterators). Checks modCount. Fail-safe: iterates on a snapshot/copy, no exception (CopyOnWriteArrayList, ConcurrentHashMap). Fail-safe uses more memory, may not reflect latest changes.

Q54: What does Collections.frequency() do?

Returns the number of times an element appears in a collection using equals(). Example: Collections.frequency(list, "java"). For Map, use Collectors.groupingBy + Collectors.counting() in streams. Also useful: Collections.disjoint(c1, c2) — true if no common elements.

Q55: Explain Collections.binarySearch() requirements and behavior.

The list MUST be sorted. Returns index of element if found; otherwise -(insertion point) - 1. Uses Comparable or Comparator. Only works correctly on sorted RandomAccess lists (ArrayList). For LinkedList it works but is O(n) due to linear traversal. Always sort before binary search.

Q56: What is the difference between Collections.emptyList() and List.of()?

Collections.emptyList(): returns a shared singleton empty list, mutable type signature but throws on add. List.of() (Java 9+): creates a truly immutable, null-free list. For non-empty lists use List.of(a,b,c). Prefer List.of() in modern Java for clarity and immutability guarantees.

Q57: How does Collections.nCopies() work?

Returns an immutable List of n copies of the specified object — all elements share the same reference. Good for initializing lists. Example: Collections.nCopies(5, 0) returns [0,0,0,0,0]. Note: since it's immutable, you can't add/remove. Use new ArrayList<>(Collections.nCopies(n, val)) for mutable version.

Q58: Explain the use of Collections.checkedCollection().

Returns a dynamically type-safe view. At runtime, throws ClassCastException if you try to add wrong type. Useful to track down heap pollution from raw types or unchecked casts. Example: Collections.checkedList(rawList, String.class) — every add is type-checked at runtime.

Q59: What are the SOLID principles and how do they apply to Collections usage?

S: Single Responsibility — collection should have one purpose. O: Open/Closed — use interface types (List, Map) not implementations. L: Liskov — ArrayList/LinkedList can substitute List. I:

Interface Segregation — use the most specific interface needed (List vs Collection). D: Dependency Inversion — depend on List<T> not ArrayList<T> in method signatures.

Q60: How to create an immutable collection in Java 9+?

List.of(1,2,3), Set.of("a","b"), Map.of("k1","v1","k2","v2"). Also: Map.copyOf(existingMap), List.copyOf(existingList). These are null-free and throw UnsupportedOperationException on mutation. For ordered Map: use Map.ofEntries(Map.entry(k,v), ...).

Section 6: Advanced & Concurrent Collections (Q61–80)

Q61: What is ConcurrentHashMap's compute() method?

compute(key, remappingFn) atomically applies the function to the current value (or null if absent) and replaces it. If fn returns null, the key is removed. Atomic — no external synchronization needed for read-modify-write patterns like counting: map.compute(word, (k,v) -> v == null ? 1 : v+1).

Q62: Explain the merge() method of Map.

map.merge(key, value, (oldVal, newVal) -> merged) — if key absent, inserts value; if present, applies the merge function. Perfect for aggregation: map.merge(word, 1, Integer::sum) — word frequency counting in one line. If merge function returns null, key is removed.

Q63: What is CopyOnWriteArrayList and its performance trade-offs?

On every add/set/remove, a new copy of the internal array is created. Reads (get, iterator, size) are lock-free and never throw ConcurrentModificationException. Write operations are O(n). Best for: event listener lists, rarely-modified read-heavy collections. Avoid for write-heavy scenarios.

Q64: How does ConcurrentSkipListMap work?

Thread-safe, sorted map using skip list data structure. O(log n) operations. Alternative to synchronized TreeMap. Supports concurrent reads and writes without locking entire structure. Key-value iteration in sorted order. Use when you need a concurrent sorted map.

Q65: What is the difference between HashMap and IdentityHashMap?

IdentityHashMap uses == (reference equality) instead of equals() for key comparison, and System.identityHashCode() instead of hashCode(). Two distinct objects with same content are treated as different keys. Use case: object graph traversal where you need to track specific instances (serialization, deep copy).

Q66: How to sort a Map by value?

entrySet().stream().sorted(Map.Entry.comparingByValue()).collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, (e1,e2)->e1, LinkedHashMap::new)). LinkedHashMap preserves insertion order, maintaining the sorted order. For reverse: Map.Entry.comparingByValue(Comparator.reverseOrder()).

Q67: What is a Deque and how does ArrayDeque replace Stack?

Deque = double-ended queue. ArrayDeque.push(e) = addFirst(e), pop() = removeFirst(). Much faster than Stack (no synchronization). Stack is a legacy class extending Vector. Java docs explicitly recommend ArrayDeque as a stack. ArrayDeque is also faster than LinkedList as a queue.

Q68: How to implement a thread-safe counter without synchronized?

Use AtomicInteger: AtomicInteger counter = new AtomicInteger(0); counter.incrementAndGet(); counter.compareAndSet(expected, update). CAS-based, no locking, very fast. LongAdder is even faster for high-contention counters — distributes increments across cells, reduces contention.

Q69: What are the differences between HashMap.forEach() and entrySet iteration?

map.forEach((k,v) -> ...) is cleaner, Java 8+, uses Consumer. entrySet() iteration is classic, allows break via iterator. forEach cannot throw checked exceptions easily. Both are O(n). ConcurrentHashMap.forEach() has additional parallelism threshold parameter for parallel execution.

Q70: How to find duplicate elements in a List efficiently?

Set<T> seen = new HashSet<>(); list.stream().filter(e -> !seen.add(e)).collect(Collectors.toList()) — seen.add() returns false for duplicates. O(n) time and space. Alternative: Collections.frequency() for single element check. For count map: Collectors.groupingBy(Function.identity(), Collectors.counting()).

Q71: What is a SortedMap and NavigableMap?

SortedMap: extends Map, keys in sorted order, firstKey()/lastKey()/headMap/tailMap/subMap. NavigableMap extends SortedMap: adds floor/ceiling/lower/higher/descendingMap/pollFirstEntry/pollLastEntry. TreeMap implements NavigableMap. ConcurrentSkipListMap is the concurrent implementation.

Q72: How to group a List into a Map<K, List<V>> using Streams?

Collectors.groupingBy(classifier): map each element to a group key, collect values. Example: Map<String, List<Employee>> = list.stream().collect(Collectors.groupingBy(Employee::getDept)). With downstream collector: groupingBy(dept, Collectors.counting()) gives Map<String, Long>.

Q73: What is the difference between List.copyOf() and new ArrayList<>(existing)?

List.copyOf() returns an unmodifiable copy — throws UnsupportedOperationException on mutation. new ArrayList<>(existing) creates a mutable copy. List.copyOf() also throws NullPointerException if source contains nulls. Use copyOf() when you want to expose a safe view to external callers.

Q74: How does PriorityBlockingQueue differ from PriorityQueue?

PriorityBlockingQueue is thread-safe and supports blocking take() operation — blocks if empty. PriorityQueue is not thread-safe. PriorityBlockingQueue is unbounded (no put() blocking). Use in thread pool implementations and task scheduling where threads must wait for the highest-priority task.

Q75: What is the collections framework's relationship to Iterable and Iterator?

Iterable<T> requires iterator() method — enables for-each loops. Iterator<T> has hasNext(), next(), remove(). Collections implement Iterable. For custom data structures: implement Iterable to support for-each, implement Iterator for traversal logic. ListIterator extends Iterator with hasPrevious/previous/add/set for bidirectional traversal.

Q76: How to efficiently find the intersection of two large lists?

Convert one list to HashSet (O(n)), then filter the other list using set.contains() (O(m)). Total: O(n+m) vs O(n*m) for nested loops. list1.stream().filter(new HashSet<>(list2)::contains).collect(Collectors.toList()). For sorted lists, two-pointer approach gives O(n+m) with O(1) space.

Q77: What is an ArrayBlockingQueue and its fairness parameter?

Bounded blocking queue backed by array. Constructor: ArrayBlockingQueue(capacity, fair). fair=true: threads acquire lock in FIFO order — prevents starvation but reduces throughput. fair=false (default): no ordering guarantee, higher throughput. Use fair=true only when thread ordering is critical to correctness.

Q78: How to implement a simple cache with automatic expiry using collections?

Use LinkedHashMap with accessOrder=true + removeEldestEntry() for LRU. For time-based expiry: wrap values in a holder with timestamp, remove in background thread or lazy removal on access. Production alternative: Caffeine cache library — high-performance caching with W-TinyLFU eviction policy.

Q79: What is Collections.rotate() and how does it work?

Collections.rotate(list, distance) — rotates elements by distance positions. Positive: right rotation (last distance elements move to front). Negative: left rotation. Efficient implementation uses three reversal algorithm: O(n). Example use: shifting elements in circular buffer or implementing rotation problems.

Q80: How do Java's collection streams interact with short-circuit operations?

Short-circuit operations (findFirst, anyMatch, allMatch, noneMatch, limit) stop processing as soon as result is determined — don't need to process all elements. findFirst() on parallel stream is O(1) effective. anyMatch(pred) returns true at first match. This makes streams efficient for search operations on large collections.

PHASE 3 · DAYS 19–26

Spring Boot Advanced

Days 19–20: Spring Core & Dependency Injection

Spring Core Topics

- IoC Container: BeanFactory vs ApplicationContext
- @Component, @Service, @Repository, @Controller stereotypes
- @Autowired: constructor injection (preferred), setter injection, field injection
- @Primary and @Qualifier for ambiguous beans
- Bean scopes: singleton (default), prototype, request, session, application
- Lifecycle: @PostConstruct, @PreDestroy, InitializingBean, DisposableBean
- @Conditional, @Profile — conditional bean creation
- Spring AOP: @Aspect, @Before, @After, @Around, Pointcut expressions

Q: What is the difference between @Component, @Service, and @Repository?

All three are specializations of @Component — functionally equivalent for component scanning. @Service: semantic marker for business layer, no extra behavior. @Repository: enables persistence exception translation (wraps JPA/JDBC exceptions into Spring DataAccessException). @Controller: marks MVC controllers. Use the right one for clarity and AOP pointcut specificity.

Days 21–22: Spring Boot Auto-Configuration & REST

Spring Boot REST API Topics

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan
- @RestController = @Controller + @ResponseBody
- HTTP mappings: @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping
- @PathVariable, @RequestParam, @RequestBody, @RequestHeader
- ResponseEntity<T> — control HTTP status codes and headers
- @Valid with Bean Validation: @NotNull, @NotBlank, @Size, @Email, @Pattern, @Min/@Max
- @ControllerAdvice + @ExceptionHandler — global exception handling
- Content negotiation, @Produces, @Consumes
- HATEOAS, HAL format for REST maturity level 3

Q: How does Spring Boot auto-configuration work?

spring.factories (Spring Boot 2) or META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (Spring Boot 3) lists auto-configuration classes. @ConditionalOnClass, @ConditionalOnMissingBean,

@ConditionalOnProperty determine if config applies. Use --debug flag to see which auto-configurations were applied or excluded.

Days 23–24: Spring Data JPA + Database

JPA & Database Topics

- Entity relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- FetchType.LAZY vs EAGER — N+1 problem and solutions (JOIN FETCH, @EntityGraph)
- JPQL, Named Queries, @Query with nativeQuery=true
- Pagination: Pageable, Page<T>, Slice<T>, @PageableDefault
- Transactional: @Transactional propagation (REQUIRED, REQUIRES_NEW, NESTED), isolation levels
- Optimistic locking: @Version, pessimistic locking: LockModeType
- Spring Data projections: interface projections, class projections, dynamic projections
- CrudRepository vs JpaRepository vs PagingAndSortingRepository
- Database migrations: Flyway vs Liquibase

Days 25–26: Spring Security + Testing

Security & Testing

- JWT authentication: filter chain, JwtAuthenticationFilter, token validation
- OAuth2/OpenID Connect basics — resource server, authorization server
- @PreAuthorize, @PostAuthorize, @Secured — method-level security
- CORS configuration: @CrossOrigin, CorsConfigurationSource
- Testing: @SpringBootTest vs @WebMvcTest vs @DataJpaTest — slice tests
- MockMvc for controller tests, @MockBean for dependency mocking
- @TestContainers — real database tests with Docker
- JUnit 5: @ExtendWith, @BeforeEach, @AfterEach, parameterized tests
- Mockito: when/thenReturn, verify, ArgumentCaptor, spy

PHASE 4 · DAYS 27–34

Microservices Architecture & Patterns

Days 27–28: Core Microservices Patterns

Pattern	Description + Use Case
API Gateway	Single entry point for all clients. Routes requests, handles auth, rate limiting, SSL termination. Spring Cloud Gateway or AWS API Gateway.
Service Discovery	Eureka (client-side), Consul, AWS Cloud Map. Services register themselves; callers look up by name, not IP.
Circuit Breaker	Resilience4j @CircuitBreaker. States: CLOSED (normal), OPEN (fail fast), HALF_OPEN (test recovery). Prevents cascade failures.
Bulkhead	Isolate failures. Separate thread pools per downstream service. @Bulkhead in Resilience4j.
Saga Pattern	Distributed transactions without 2PC. Choreography (events) or Orchestration (saga orchestrator). Compensating transactions on failure.
CQRS	Command Query Responsibility Segregation. Separate read/write models. Read side can be denormalized, optimized for queries.
Event Sourcing	Store state as sequence of events, not current snapshot. Replay events to rebuild state. Pairs well with CQRS.
Strangler Fig	Gradually migrate monolith to microservices by intercepting calls and routing to new services.
Sidecar	Deploy helper process alongside main container (e.g., Envoy proxy). Handles logging, monitoring, service mesh.
Outbox Pattern	Reliably publish events: write to DB outbox table in same transaction, then publish to Kafka via CDC/polling. Avoids dual-write problem.

Days 29–30: Inter-Service Communication

Communication Patterns

- Synchronous: REST (HTTP/HTTPS) via WebClient (reactive) or RestTemplate (legacy)
- Asynchronous: Kafka, RabbitMQ, AWS SQS for event-driven communication

- gRPC: binary protocol, protobuf, faster than REST, good for internal services
- WebClient with retry, timeout, circuit breaker: Resilience4j + Spring Cloud
- Service mesh: Istio, Linkerd — infrastructure-level mTLS, tracing, circuit breaking
- Distributed tracing: Spring Cloud Sleuth + Zipkin / AWS X-Ray for trace propagation
- Correlation IDs: propagate across services for debugging (MDC in logging)

Q: How do you handle distributed transactions in microservices?

Avoid distributed transactions if possible — design for eventual consistency. When needed: 1) Saga pattern with compensating transactions. 2) Outbox pattern with Kafka for at-least-once event delivery. 3) Idempotency keys on consumers to handle duplicate messages. 2PC (Two-Phase Commit) is generally avoided — blocks and creates tight coupling. Tools: Axon Framework, Eventuate for saga orchestration.

Days 31–32: Resilience & Observability

Production Readiness Topics

- Resilience4j: @CircuitBreaker, @Retry, @RateLimiter, @TimeLimiter, @Bulkhead
- Health checks: Spring Actuator /health, /metrics, /info, Kubernetes liveness/readiness probes
- Distributed tracing: Micrometer Tracing (Spring Boot 3), Sleuth (Spring Boot 2), trace/span IDs
- Metrics: Micrometer with Prometheus + Grafana dashboards
- Centralized logging: ELK Stack (Elasticsearch, Logstash, Kibana) or AWS CloudWatch Logs Insights
- Alerting: CloudWatch Alarms, PagerDuty integration
- Rate limiting: Token bucket algorithm, leaky bucket, Redis-based rate limiting

Days 33–34: Docker, Kubernetes & CI/CD

Container & DevOps

- Docker: Dockerfile, multi-stage builds, .dockerignore, docker-compose for local dev
- Kubernetes basics: Pod, Deployment, Service, ConfigMap, Secret, Ingress
- K8s patterns: sidecar, init containers, liveness/readiness probes, resource limits
- Helm charts: templating K8s manifests
- CI/CD: Jenkins pipelines (you already have this), GitHub Actions, branching strategies (GitFlow, trunk-based)
- Blue-Green deployment vs Canary deployment vs Rolling update
- Container security: non-root user, read-only filesystem, image scanning (Trivy)

PHASE 5 · DAYS 35–42

Apache Kafka Deep Dive

Days 35–36: Kafka Core Concepts

Kafka Fundamentals

- Topics, Partitions, Offsets — the fundamental unit of storage
- Producer: key-based partitioning, round-robin, sticky partition strategy
- Consumer Groups: each partition consumed by exactly one consumer per group
- Broker, Leader/Follower replicas, ISR (In-Sync Replicas)
- ZooKeeper (legacy) vs KRaft (Kafka 3.x, no ZooKeeper dependency)
- Log compaction: retain only latest value per key — useful for event sourcing
- Kafka Connect: source/sink connectors for CDC, database integration
- Kafka Streams: stream processing library for aggregations, joins, windowing

Days 37–38: Kafka with Spring Boot

Spring Kafka Configuration

- @KafkaListener: topics, groupId, containerFactory, concurrency, batch mode
- @KafkaHandler for polymorphic message handling
- KafkaTemplate.send() — synchronous (get()) and asynchronous (ListenableFuture/CompletableFuture)
- Error handling: SeekToCurrentErrorHandler, DeadLetterPublishingRecoverer, @RetryableTopic
- Dead Letter Topic (DLT): failed messages routed to <topic>.DLT after retry exhaustion
- Consumer offset management: auto-commit vs manual commit (acknowledge())
- Exactly-once semantics: transactional producer, read_committed isolation level
- Schema Registry: Apache Avro + Confluent Schema Registry for schema evolution
- Embedded Kafka for testing: @EmbeddedKafka annotation

Q: Explain Kafka's delivery guarantees: at-most-once, at-least-once, exactly-once.

At-most-once: producer fire-and-forget, consumer auto-commit before processing — possible message loss. At-least-once: producer retries with acks=all, consumer manual commit after processing — possible duplicates. Exactly-once: requires idempotent producer (enable.idempotence=true) + transactional API + read_committed on consumer. EOS is expensive — use at-least-once + idempotent consumers for most cases.

Q: What is consumer lag and how do you monitor it?

Consumer lag = latest offset (LEO) - consumer committed offset. High lag means consumers are behind. Monitor with: kafka-consumer-groups.sh --describe, Burrow (LinkedIn), or Prometheus JMX exporter + Grafana. CloudWatch has kafka_consumer_lag metric for MSK. Alert when lag exceeds threshold. Fix: increase partitions, add consumers (up to partition count), optimize processing.

Days 39–40: Kafka Advanced Topics

Topic	Key Points to Know
Partition Rebalancing	Triggered when consumer joins/leaves group. Static membership (group.instance.id) reduces rebalancing in K8s.
Message Ordering	Guaranteed within a partition, not across partitions. Use same key for ordered messages. Keyed producer routes same key to same partition.
Compacted Topics	Only last value per key retained. Delete marker (null value) = tombstone. Good for state snapshots, reference data.
Kafka Transactions	Atomic produce to multiple topics/partitions. transactional.id, beginTransaction, commitTransaction, abortTransaction.
Kafka Streams	Stateless: map, filter, flatMap. Stateful: aggregate, join, windowing (tumbling, hopping, session windows). State stores backed by RocksDB + changelog topic.
MSK (AWS)	Managed Kafka. Auto-scaling storage, MSK Connect for connectors, MSK Serverless for variable workloads, CloudWatch integration.

Days 41–42: Kafka Interview Prep

Q: How do you ensure message ordering in Kafka?

Use a partition key — all messages with the same key go to the same partition and are consumed in order. max.in.flight.requests.per.connection=1 ensures order even on retry (or use idempotent producer which handles this). If global ordering is needed across partitions, it means a single partition — which limits throughput.

Q: What happens when a Kafka consumer crashes?

Consumer group coordinator detects missed heartbeats after session.timeout.ms. Triggers a rebalance — partitions are redistributed to remaining consumers. With auto-commit, in-flight messages since last commit may be reprocessed. With manual commit, only after successful processing is offset committed — reduces data loss but may cause duplicates on restart.

PHASE 6 · DAYS 43–50

AWS Deep Dive

Days 43–44: Amazon S3

S3 Core Concepts

- Bucket policies vs IAM policies vs ACLs — when each applies
- Storage classes: Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant/Flexible/Deep Archive
- S3 lifecycle policies: transition rules, expiration rules
- Versioning, MFA Delete, Object Lock (WORM compliance)
- Server-side encryption: SSE-S3 (AES-256), SSE-KMS (audit trail), SSE-C (customer key)
- Client-side encryption with AWS Encryption SDK
- Multipart upload for files > 100MB — S3AsyncClient (you've used this at Deloitte)
- S3 Event Notifications: Lambda, SQS, SNS triggers on object creation/deletion
- S3 Transfer Acceleration: CloudFront edge for global upload speed
- Presigned URLs: temporary access without AWS credentials (expires in N seconds)
- S3 Access Points: named network endpoints with individual policies
- S3 Select + Glacier Select: SQL queries on objects without full download

Q: How do you securely share S3 files with external users without making the bucket public?

Generate presigned URLs using AWS SDK: s3Client.generatePresignedUrl(bucket, key, expiry). URLs are time-limited (max 7 days with IAM role, max 12 hours with temporary credentials). Use S3 presigned POST for direct client uploads. Never make bucket public — use bucket policies with resource-based conditions for specific accounts.

Days 45–46: EC2, ECS & Compute

Compute Services

- EC2 instance types: General Purpose (t3/m6), Compute (c6), Memory (r6), Storage (i3)
- EC2 pricing: On-Demand, Reserved (1yr/3yr), Spot (up to 90% cheaper, can be interrupted), Savings Plans
- Auto Scaling Groups: scaling policies (Target Tracking, Step, Scheduled), health checks, warm pools
- Launch Templates vs Launch Configurations (always use Templates now)
- ECS: Task Definitions, Services, Clusters, Fargate vs EC2 launch type
- ECS Task Role vs Execution Role — critical distinction
- AWS Lambda: cold start, warm start, concurrency limits, reserved concurrency, provisioned concurrency
- ALB vs NLB: Application Load Balancer (L7, content-based routing) vs Network LB (L4, ultra-low latency)
- Target groups, health checks, sticky sessions, SSL/TLS termination

Days 47–48: CloudWatch, Monitoring & Security

CloudWatch & Security

- CloudWatch Metrics: namespaces, dimensions, resolution (1s vs 60s), custom metrics via SDK
- CloudWatch Logs: log groups, log streams, log insights queries (SQL-like), metric filters
- CloudWatch Alarms: thresholds, anomaly detection, composite alarms
- CloudWatch Dashboards: widgets, automatic dashboards for AWS services
- AWS X-Ray: distributed tracing, trace map, segments/subsegments, sampling
- AWS IAM: users, groups, roles, policies (identity-based vs resource-based)
- IAM best practices: least privilege, no root user, MFA, rotation, cross-account roles
- AWS KMS: CMKs, key policies, envelope encryption
- AWS Secrets Manager vs Parameter Store — cost/features trade-offs
- VPC: subnets (public/private), route tables, NAT Gateway, Security Groups vs NACLs

Days 49–50: Terraform (IaC)

Terraform Core Concepts

- HCL syntax: resource, data source, variable, output, local, module
- Terraform workflow: init → validate → plan → apply → destroy
- State: local vs remote (S3 + DynamoDB lock), state isolation with workspaces
- Modules: reusable configurations, module sources (registry, git, local path)
- Count vs for_each — prefer for_each (stable addressing, handles deletions well)
- Lifecycle rules: create_before_destroy, prevent_destroy, ignore_changes
- Data sources: querying existing infrastructure without managing it
- Dynamic blocks: generate repeated nested blocks programmatically
- Terraform import: bring existing resources under Terraform management
- Terragrunt: DRY Terraform configurations for multi-environment setups

Example: EC2 + Security Group in Terraform

```
resource "aws_instance" "app" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
  subnet_id     = aws_subnet.private.id
  iam_instance_profile = aws_iam_instance_profile.app.name
  tags = { Name = "${var.env}-app-server" }
}

resource "aws_security_group" "app" {
  ingress { from_port=8080 to_port=8080 protocol="tcp" cidr_blocks=["10.0.0.0/8"] }
  egress  { from_port=0   to_port=0    protocol="-1"  cidr_blocks=["0.0.0.0/0"] }
}
```

PHASE 7 · DAYS 51–56

SQL Mastery + Design Patterns

Days 51–52: SQL Deep Dive

SQL Topics You Must Know

- Window functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()
- OVER (PARTITION BY ... ORDER BY ...) clause — the most tested SQL topic
- CTEs (WITH clause): recursive CTEs for hierarchical data
- CASE WHEN THEN ELSE END — conditional logic
- Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF JOIN with examples
- GROUP BY with HAVING vs WHERE — HAVING filters after aggregation
- EXISTS vs IN vs JOIN — performance differences (prefer EXISTS for correlated subqueries)
- Index types: B-tree (range queries), Hash (equality), Composite index column order
- EXPLAIN / EXPLAIN ANALYZE — reading query plans, identifying sequential scans
- Transactions: ACID properties, isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable)
- PostgreSQL specifics: JSONB, array types, pg_stat_activity, VACUUM, ANALYZE

Window Functions — Most Asked SQL Interview Topic

```
-- Find the 2nd highest salary per department
SELECT * FROM (
  SELECT emp_name, dept, salary,
    DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
  FROM employees
) ranked WHERE rnk = 2;

-- Running total
SELECT date, amount, SUM(amount) OVER (ORDER BY date) AS running_total FROM sales;

-- Previous row value (LAG)
SELECT date, amount, LAG(amount) OVER (ORDER BY date) AS prev_amount FROM sales;
```

Days 53–54: Design Patterns

Pattern	Use Case & Key Point
Singleton	One instance per JVM. Thread-safe: double-checked locking with volatile, or enum Singleton (best).
Factory Method	Subclasses decide which class to instantiate. Spring uses extensively for bean creation.
Abstract Factory	Creates families of related objects. DB driver factories, UI toolkit factories.
Builder	Construct complex objects step-by-step. Lombok @Builder, StringBuilder. Avoid telescoping constructors.
Prototype	Clone objects instead of new. Java Cloneable (prefer copy constructor). Spring prototype scope.
Adapter	Convert interface of a class to another. Wrapping legacy code. RestTemplate as adapter.
Decorator	Add behavior without subclassing. Java I/O streams. Wrap ResponseEntity.
Proxy	Control access to object. Spring AOP, @Transactional, @Cacheable use JDK dynamic proxy/CGLIB.
Observer	Notify dependents on state change. Spring ApplicationEventPublisher, Kafka events.
Strategy	Encapsulate algorithm family, make interchangeable. Sort comparators, payment strategies.
Command	Encapsulate request as object. Undo/redo, job queue, Spring @Scheduled tasks.
Template Method	Define algorithm skeleton, let subclasses fill steps. JdbcTemplate, RestTemplate.

Days 55–56: System Design Fundamentals

System Design Topics (Must Know)

- CAP Theorem: Consistency, Availability, Partition Tolerance — pick 2
- PACELC extension: even without partition, trade-off between latency and consistency
- Database scaling: vertical (bigger machine) vs horizontal (sharding, read replicas)
- Sharding strategies: range-based, hash-based, directory-based
- Caching: CDN, Redis (cache-aside, write-through, write-behind), cache invalidation strategies
- Rate limiting: token bucket, leaky bucket, fixed window, sliding window log
- Load balancing: round-robin, least connections, consistent hashing

- Message queues vs event streaming: when to use SQS vs Kafka
- Consistent hashing: minimizes remapping when nodes added/removed (used in Redis Cluster, DynamoDB)
- Bloom filters: probabilistic, no false negatives, used for set membership with space savings

PHASE 8 · DAYS 57–60

Mock Interviews + System Design + Final Polish

Day 57: System Design Practice Problems

Practice These Systems (In Order of Frequency)

- URL Shortener (TinyURL): hashing, collision handling, redirect, analytics, DB choice
- Real-Time Payment System (you built at Infosys — you can speak to this): idempotency, ACID, throughput
- Notification System: fan-out, push vs pull, priority queues, rate limiting
- Distributed File Storage (like S3): consistent hashing, replication, metadata server
- Chat Application: WebSocket, message ordering, presence, delivery receipts
- Rate Limiter: token bucket, Redis implementation, distributed rate limiting
- Search Autocomplete: Trie, TypeAhead, ranking by frequency
- API Gateway / Service Mesh: routing, load balancing, circuit breaking

Day 58: Behavioral Questions (STAR Method)

Question	Your Story Angle (From Resume)
Tell me about a time you improved system performance.	Infosys VISA: reduced processing time by 4 mins per operation.
Describe a challenging technical problem you solved.	JP Morgan: multi-region API communication across us-east-1 and us-west-2.
How do you handle conflicting priorities?	Agile sprints at Deloitte — sprint planning, code reviews.
Tell me about a time you learned a new technology quickly.	WebFlux/WebClient for reactive programming at Deloitte.
Describe a time you received critical feedback.	Star Performer + Client Excellence award — shows positive track record.
How do you ensure code quality?	CI/CD with Jenkins, Postman/Swagger API testing, code reviews.

Day 59: Resume + LinkedIn Polish

Action Items for Day 59

- Quantify every bullet: 'Reduced processing time by 4 mins' is good — add volume: 'for 50K daily transactions'
- Add Kafka section: 'Implemented event-driven architecture using Apache Kafka for customer event processing'
- Add Terraform: 'Provisioned AWS EC2, RDS, IAM using Terraform IaC modules'

- Update LinkedIn: mirror resume, add Skills section (Java, Spring Boot, Kafka, AWS, Microservices, Terraform)
- Get 5+ recommendations on LinkedIn — ask colleagues from Infosys/Deloitte now
- Add a GitHub profile: push 2-3 Spring Boot + Kafka projects (microservices demo is ideal)
- Prepare your 60-second elevator pitch: background → what you do → what you're looking for

Day 60: Final Mock Interview

Last Day Checklist

Do a full mock interview (1 hr)	Review your top 10 weak areas
Practice system design out loud	Review all 80 Collections Q&A
Prepare 3 good questions to ask	Confirm your ideal target role/companies

Bonus: Additional Skills to Strengthen Your Profile

Skill	Why It Matters + What to Learn
Reactive Programming (WebFlux)	You're already using WebClient at Deloitte. Learn Project Reactor: Mono, Flux, operators, backpressure, error handling, blocking vs non-blocking.
Redis	Cache-aside pattern, distributed locks (Redisson), session storage, rate limiting. Spring Data Redis, RedisTemplate, @Cacheable.
gRPC	Faster than REST for internal microservices. Protocol Buffers, service definition, Spring gRPC starter.
GitHub Actions	CI/CD alternative to Jenkins. YAML workflows, matrix builds, secrets, artifacts. Very popular in new companies.
OpenTelemetry	Standard for traces+metrics+logs. Replaces Sleuth in Spring Boot 3. Auto-instrumentation agents.
Docker Multi-stage Builds	Reduce image size from 800MB to 80MB. Separate build and runtime stages. .dockerignore best practices.

Skill	Why It Matters + What to Learn
API Design Best Practices	REST maturity levels, versioning strategies (URL vs header), pagination (cursor vs offset), error response standards (RFC 7807).
Cloud-Native Patterns	12-Factor App methodology, configuration via env vars, stateless services, health endpoints for K8s probes.

You have the experience. This plan gives you the language and depth to articulate it in interviews.

60 days. 1 hour per day minimum. Consistent beats intense. Good luck, Zuber!